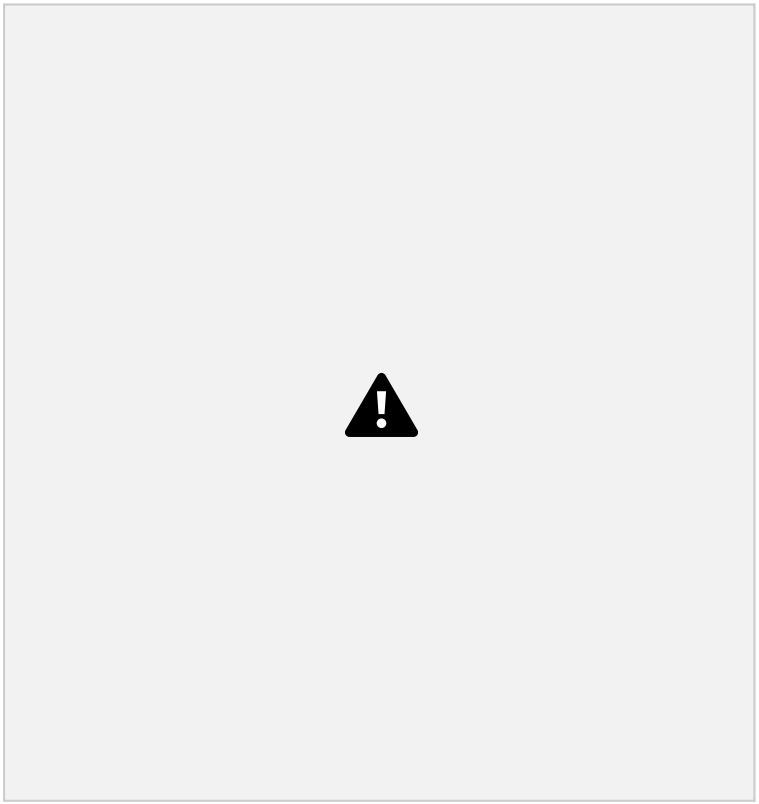
We promise that this tutorial is not a permanent and unchanging document. We will continuously revise and optimize the tutorial based on everyone's feedback and actual development experience.

Preparation work and precautions

Software environment

Software Development Environment based on Vivado 2020.1



Hardware environment

Development board model	Chip model
XU4EV-P	czu4ev-sfvc784-1-i
XU5EV-P	azu5ev-sfvc784-1-i

Batch Download QSPI Flash

All project directories have a bootimage folder, which stores the corresponding BOOT.bin file. You can copy this file to the Vitis_image_download folder and overwrite the original BOOT.bin. You can also put BOOT.bin on the SD card to start the verification function

vitis_image_download folder under the course_s2 directory, enter the folder, right-click program_qspi bat, open edit

Change the program_flash path to your own software installation path, save and close. Double-click program_qspi to download BOOT. BIN to QSPI FLASH. It is recommended to download in JTAG mode. You can also use the SD card boot method to copy the BOOT.bin file to the SD to boot. ## Batch build Vitis project

Due to the large space occupied by the Vitis project after compilation, in order to save everyone's valuable time, we provide batch processing tcl scripts for the Vitis project.

There is a vitis folder under each project, which contains the hardware description file xx.xsa and the script for automatically creating the project

All you need to do is right-click and edit the build_vitis .bat file in the auto_create_vitis folder (Note: the source file of the vitis program that does not use the template is in the folder in the vitisauto_create_vitissrc)

Replace the xsct.bat path in the yellow box with your own installation path, which is xx\ Vitis\ 2020.1\ bin\ xsct.bat

After saving, double-click build_vitis bat to create the project

Compile finished, press any key to exit

Open the Vitis software, select the project path, and launch. After opening, close the Welcom interface Project can be used

Starting from this chapter, FPGA engineers and software development engineers will collaborate to implement it.

The previous experiments were all conducted on the PL side, which shows no difference from the ordinary FPGA development process. The main advantage of ZYNQ is the reasonable combination of FPGA and ARM, which puts higher requirements on developers. Starting from this chapter, we will use ARM, which is what we call PS. In this chapter, we will use a simple serial port printing to experience the characteristics of Vivado Vitis and PS side.

The previous experiments are all things that FPGA engineers should do. Starting from this chapter, there is division of labor. FPGA engineers are responsible for building the Vivado project and providing hardware to software developers, who can then develop applications based on this. Good division of labor is also beneficial for project progress. If software developers want to do everything, they may need to spend a lot of time and energy learning about FPGA knowledge. It is a painful process to switch from software thinking to hardware thinking. If it is purely for learning and there is time, it is another matter. Professional people doing professional things is a good choice.

1.

Hardware introduction

We can see from the schematic that the ZYNQ chip is divided into PL and PS. The IO allocation on the PS side is relatively fixed and cannot be arbitrarily allocated. Moreover, there is no need to allocate pins in the Vivado software. Although this experiment only uses PS, a Vivado project needs to be established to configure PS pins. Although the ARM on the PS side is hardcore, the ARM hardcore needs to be added to the project in ZYNQ before it can be used. The previous chapters introduced the code form of the project, and this chapter begins to introduce the graphical way to establish the project in ZYNQ.

Job responsibilities of FPGA engineer

The following introduces the responsibilities of FPGA engineers.

2.

Vivado Project Establishment

1.

Create a project named "ps_hello", the establishment process will not be repeated, refer to "PL's" Hello World "LED experiment".

2.

Click "Create Block Design" to create a Block design, which is a graphical design

3.

"Design name" is not modified here, keep the default "design_1", you can modify it as needed, but the name should be as short as possible, otherwise there will be problems with compiling under Windows.

4.

Click the "Add IP" shortcut icon

5.

Search for "zynq", double-click "Zynq UltraScale + MPSoC" in the search results list.

6.

Double-click the ZYNQ core in the Block diagram and configure the relevant parameters

7.

The first interface that appears is the architecture diagram of ZYNQ hardcore, which can be clearly seen. You can refer to the ug1085 document for a detailed introduction to ZYNQ. The green part in the figure is the configurable module, which can be clicked to enter the corresponding editing interface. Of course, you can also enter the editing in the left window. Below, the functions of each window are introduced one by one.

2.1

Low Speed Configuration

1.

In the I/O Configuration window, configure the voltages of BANK0~ BANK2 as LVCMOS18 and BANK3 as LVCMOS33. First, configure the Low Speed pin, check QSPI, set it to "Single" mode, set the Data Mode to "x4", and check Feedback Clk.

2.

Check SD 0 to configure eMMC. Select MIO13.. 22, select eMMC for Slot Type, set Data Transfer Mode to 8Bit, check Reset, and select MIO23.

3.

Check SD 1 to configure SD card. Select MIO 46.. 51, Slot Type select SD 2.0, Data Transfer Mode select 4Bit, check CD to detect SD card insertion, select MIO45

4.

Check CAN 0, select MIO 38.. 39, check CAN 1, select MIO 32.. 33

5.

Check I2C 1, I2C for EEPROM, etc., select MIO 24.. 25

6.

Check serial port UART 0, select MIO 42.. 43, check GPIO1 MIO

7.

Check TTC0~ TTC 3

2.2

High Speed Configuration

1.

For the High Speed section, first configure the PS side Ethernet, check GEM 3, select MIO 64.. 75, check MDIO 3, select MIO 76.. 77.

2.2

High Speed Configuration

1.

For the High Speed section, first configure the PS side Ethernet, check GEM 3, select MIO 64.. 75, check MDIO 3, select MIO 76.. 77.

2.

Check USB 0, select MIO 52.. 63, check USB 3.0, select GT Lane1

USB Reset Select MIO 31

3.

Check PCIe

4.

Click on Switch To Advanced Mode, select PCIe Configuration, modify the following parameters, and configure it to ROOT mode

5.

Return to I/O Configuration, reset and select MIO 37; check Display Port, select MIO 27.. 30, and select Dual Higher for Lane Selection.

At this point, the I/O part is configured

2.3

Clock configuration

1.

In the Clock Configuration interface, the Input Clocks window configures the reference clock, where the reference clock PSS_REF_CLOCK for ARM defaults to 33.333MHz; PCIe selects Ref Clk0, 100MHz; Display Port selects Ref Clk2, 27MHz; USB0 selects Ref Clk1, 26MHz.

2.

In the Output Clocks window, if it is not IOPLL, change it to IOPLL to maintain consistency and use the same PLL.

3.

PL's clock remains default, which is the clock provided to the PL-side logic.

4.

Full Power section, others remain default, change DP_VIDEO to VPLL, DP_AUDIO and DP_STC to RPLL.

Modify the bottom Interconnect as follows

Other parts remain default, so the clock configuration is complete.

2.4

DDR configuration

In the DDR Configuration window, Load DDR Presets select "DDR4_MICRON_MT40A256M16GE_083E"

Modify the parameters as follows Leave the others as default, click OK, the configuration is complete, and connect the clock as follows: 8.

Select Block Design, right-click "Create HDL Wrapper...", create a Verilog or VHDL file, and generate an HDL top-level file for Block Design.

9.

Keep the default options and click "OK".

10.

Expand the design and you can see that PS is used as a regular IP.

11.

Select the block design, right-click on "Generate Output Products", this step will generate the output file of the block, including IP, instantiated templates, RTL source files, XDC constraints, third-party comprehensive source files, etc. For subsequent operations.

12.

Click "Generate"

13.

Export hardware information in the menu bar "File - > Export - > Export Hardware...", which includes configuration information for the PS side.

14.

Select Fixed in the pop-up window and click Next.

15.

Click "OK" in the pop-up dialog box. Since the experiment only uses the PS serial port and does not require PL participation, there is no enable to not select "Include bitstream". Click Next

16.

You can modify the export name and export path. By default, it is located in the vivado project directory. This file can be placed in a suitable location according to your needs. It does not have to be placed under the vivado project. Vivado and vitis software are independent. Here we choose to leave the default unchanged. Click Next.

Click Finish At this point, you can see the xsa file in the project directory. This file contains information about the Vivado hardware design and can be used by software developers. At this point, the work of the FPGA engineer comes to an end.

Job responsibilities of software engineers

The Vitis project directory is "ps_hello/vitis"

The following is the content that software engineers are responsible for.

3.

Debugging Vitis

3.1

Create Application Project

1.

Create a new folder and copy the xx.xsa file exported from vivado.

2.

Vitis is an independent software that can be opened by double-clicking the Vitis software

You can also open the Vitis software by selecting ToolsLaunch Vitis in the Vivado software Select the newly created folder and click "Launch". 3.

After starting Vitis, the interface is as follows. Click "Create Application Project". This option will generate the APP project and Platfrom project. The Platform project is similar to the previous version of the hardware platform, including relevant files for hardware support and BSP.

4.

The first page is the introduction page, skip directly and click Next.

5.

Select "Create a new platform from hardware (XSA) " and select "Browse".

Select the previously generated xsa and click Open. 6.

The Generate boot components option at the bottom, if checked, the software will automatically generate the fsbl project. We usually choose to check it by default.

7.

Fill in the APP project name, click on the box to select the corresponding processor, we keep the default here

8.

In this interface, you can modify the Domain name, select the operating system, ARM architecture, etc. Here, the default is maintained, and the operating system is selected as standalone, which means bare metal.

9.

Select the "Hellow World" template and click "Finish" to complete

10.

After completion, you can see that two projects have been generated, one is the hardware platform project, which is the Platfrom project mentioned earlier, and the other is the APP project

11.

After expanding the Platform project, you can see that it contains the BSP project and the zynq_fsbl project (this project is the result of selecting Generate boot components). Double-click platform.spr to see the corresponding BSP project generated by the Platform, where you can configure the BSP. Software developers are more clear that BSP means Board Support Package, which contains the driver files needed for development and is used for application development. You can see that there are multiple BSPs under the Platform, which is different from the previous SDK software. zynqmp_fsbl is the BSP of fsbl, and domain_psu_cortexa53_0 is the BSP of the APP project. You can also add BSP in the Platform, which will be discussed in future routines.

12.

Click on BSP to see the peripheral drivers provided by the project. Documentation is the documentation for the drivers provided by xilinx, and Import Examples is the example project provided by xilinx to speed up learning.

13.

Select the APP project, right-click Build Project, or click the "Hammer" button in the menu bar to compile the project.

13.

Select the APP project, right-click Build Project, or click the "Hammer" button in the menu bar to compile the project.

Compile finished, generate elf file 15.

Connect JTAG cable to development board, UART USB cable to PC

16.

Using PuTTY software as a serial port end point debugging tool, PuTTY is a small software that does not require installation

17.

Select Serial, fill in COM3 for Serial line, fill in 115200 for Speed, fill in the COM3 string slogan according to the display in Facility Management, click "Open".

18.

Before powering on, it is best to set the startup mode of the development board to JTAG mode and unplug it to the "ON" position

19.

Power on the development board and prepare to run the program. The development board comes with the program when it leaves the factory. Here, you can select the JTAG mode for the running mode , and then power it back on . Select "hello", right-click, and you can see many options. This experiment will use "Run as" here, which means to run the program. There are many options in "Run as". Select the first "Launch on Hardware (Single Application Debug) " and use system debugging to run the program directly.

20.

At this time, by observing the serial port software, you can see the output "Hello World".

21.

To ensure reliable debugging of the system, it is best to right-click "Run As - > Run Configuration..."

22.

We can take a look at the configuration inside. Reset entire system is selected by default, which is different from previous SDK software. If there is also PL design in the system, "Program FPGA" must also be selected.

23.

In addition to "Run As", you can also "Debug As", which allows you to set breakpoints and run step by step

24.

Enter Debug mode

25.

Like other C language development IDEs, it can be run step by step, set breakpoints, etc

26.

You can switch IDE mode in the upper right corner.

4.

Solidification procedure

Ordinary FPGAs can generally be started from flash or passively loaded. The startup of ZYNQ is dominated by ARM, including the loading of FPGA programs. The startup of ZYNQ MPSoC generally consists of three steps, which are also introduced in UG1085.

Pre-configuration satge: The preloading phase is controlled by the PMU and executes the code setting system in the PMU ROM. The PMU handles all reset and wake-up processes.

Configuration stage: The next step is the most important one. After BootRom (part of the CSU ROM code) transfers FSBL to OCM, the processor starts executing FSBL code. FSBL mainly has the following functions:

- Initialize PS side configuration, MIO, PLL, DDR, QSPI, SD, etc
- If there is a PL-side program, load the PL-side bitstream.
- Move user programs to DDR and jump to execution

After FSBL starts executing, the CSU ROM code enters the post-configuration stage, responsible for system intervention response. CSU provides continuous hardware support for verifying file correctness, loading PL through PCAP, storing and managing security keys, decryption, etc.

4.1

Generate FSBL

FSBL is a secondary bootstrap program that completes MIO allocation, clock, PLL, DDR controller initialization, SD, QSPI controller initialization, searches for bitstream configuration FPGA through startup mode, then searches for user programs to load into DDR, and finally hands over to the application program for execution.

1.

Since the Generate boot components option was selected when creating a new one, the Platform has imported the fsbl project and generated the corresponding elf file.

2.

Modify the debug macro definition FSBL_DEBUG_INFO_VAL, you can output some status information of FSBL at startup, which is conducive to debugging, but it will lead to a longer startup time. Save the file. You can take a look at the files of many peripherals in fsbl, including psu_init, qspi, sd, etc. You can read the code carefully. Of course, this fsbl template can also be modified. As for how to modify it according to your own needs.

3.

Rebuild Project

4.

Next, we can click on the system of the APP project, right-click and select Build project.

5.

At this time, an additional Debug folder will appear, and the corresponding BOOT. BIN will be generated.

6.

Another method is to right-click on the system of the APP project and select Creat Boot Image. In the pop-up window, you can see the generated BIF file path. The BIF

file is the configuration file for generating BOOT files, and there is also the generated BOOT.bin file path. The BOOT.bin file is the startup file we need, which can be placed on an SD card for startup or burned to QSPI Flash.

7.

There are files to be synthesized in the Boot image partitions list. The first file must be the bootloader file, which is the fsbl.elf file generated above. The second file is the FPGA configuration file bitstream, which does not need to be added in this experiment because there is no bitstream for FPGA. The third file is the application program, which is hello.elf in this experiment. Since there is no bitstream, only the bootloader and application program will be added in this experiment. Click Create Image to generate.

8.

The BOOT.bin file can be found in the generated directory

4.2

SD card boot test

1.

Format the SD card, it can only be formatted as FAT32 format, other formats cannot be started

2.

Put the BOOT.bin file in the root directory

3.

SD card inserted into the SD card slot of the development board

4.

Adjust the startup mode to SD card startup

5.

Open the serial port software, power on, and you can see the printing information. The red box is the FSBL startup information, and the yellow arrow part is the executed application helloworld.

4.3

QSPI startup test

1.

Vitis menu Xilinx - > Program Flash

2.

Select the latest Hardware Platform, select the BOOT.bin file to be burned for the Image FIle file, and select fsbl.elf for the FSBL file. Select Verify after flash and verify the flash after burning is complete.

3.

Click Program and wait for the burning to complete

4.

Set the startup mode to QSPI, restart, and you can see the same startup effect as SD in the serial port software.

4.4

Burning QSPI under Vivado

1.

Select the device under HARDWARE MANGER, right-click Add Configuration Memory Device.

2.

Select Try Micron, select qspi as the type, x4-single as the width, and 256 as the Density. When wt25qu256 appears, select the model with the red box.

3.

Right-click and select the programming file

4.

Select the file and fsbl file to burn, and you can burn. If the burning is not in JTAG startup mode, the software will give a warning, so it is recommended to set the QSPI to JTAG startup mode when burning

4.5

Quickly burn QSPI using batch files.

1.

Create a new program_qspi .txt text file with the extension changed to bat, fill in the content as follows,

E:\ XilinxVitis\ Vitis\ 2020.1\ bin\ program_flash is our tool path, modify it appropriately according to the installation path, -f is the file to be burned, -fsbl is the fsbl file to be burned, and -verify is the verification option.

```
allE:\XilinxVitis\Vitis\2020.1\bin\program_flash -f BOOT.bin -offset 0 -flash_type qspi-x4-single
'sbl fsbl.elf -verify

ause
```

2.

Put the BOOT.bin, fsbl, and bat files to be burned together

3.

After plugging in the JTAG cable and powering it on, double-click the bat file to burn the flash.

If you want to generate a BOOT. BIN file, you can choose system to compile. In this case, both elf and BOOT. BIN will be generated. I suffered losses when I first used it. Every time I compile, I choose system. As a result, I have to wait for BOOT. BIN to be generated every time, which wastes time. Everyone can pay attention.

7.

Summary of this chapter

6.

Click on platform.spr and click on BSP

7.

Find the RTC driver and click Import Examples

8.

Very lucky, there is an example of interruption. How do you know this example is an example of interruption? It is guessed through "intr", so basic skills are very important, otherwise you won't even know how to find routines.

2.

The base address, device ID, interrupt, etc. of each peripheral are defined in the xparameters.h Header File

For example, the DEVICE_ID macro definition in the program is found in this file.

3.

In the libsrc folder, there are definitions of peripheral functions and instructions for use

7.

Click Run

8.

The program runs to this point, and then sets a breakpoint at the RecvFrame

9.

Click Run again to see the data generated by TxFrame.

10.

Press F6 to run step by step, and you can see the data received by RxFrame, thus completing the data transmission and reception from CAN0 to CAN1.

3.

On board verification

1.

First, you need to save a 1920 * 1080 pixel, 24-bit BMP file to the SD card, with the file name 1.bmp (the file is in the project directory). If the development board is powered off, insert the SD card into the card holder.

2.

Connect the development board to the DP monitor, then power it on. After running the download program, we can display the image of the 1.bmp file stored in the SD card on the DP monitor.

3.

Afterwards, you can power off the development board, insert the SD card into the computer, and you can see more CAT. BMP.

Part 9: Lwip for PS-side Ethernet

The vivado project directory is "ps_hello/vivado"

The vitis project directory is "ps_net/vitis"

4.

When a character is input, the development board returns the same character

3.

Experimental summary

Through experiments, we have a deeper understanding of the development of Vitis programs. This experiment simply explains how to create an LWIP application, which can complete protocols such as UDP and TCP. In subsequent tutorials, we will provide specific applications based on Ethernet, such as ADC collecting data and sending it

through Ethernet, and camera data being sent to the host computer for display through Ethernet.

Part 10: Network remote update QSPI on PS side

The vivado project directory is "ps_hello/vivado"

The vitis project directory is "ps_remote/vitis"

In actual work, there may be product upgrade issues. If you follow the program burning method, you may need to open the product shell, which is undoubtedly necessary. This chapter introduces a method of remotely updating FLASH programs through the network, including two methods: UDP and TCP.

Job responsibilities of software engineers

The following is the content that software engineers are responsible for.

Note: Due to the use of lwip, it is necessary to modify the lwip library before conducting this experiment. Please refer to the chapter "Lwip for PS Ethernet Use".

7.

Add some content to emio.xdc, the port name must be consistent with the top-level file port.

```
##################Compress Bitstream#############################

et_property BITSTREAM.GENERAL.COMPRESS TRUE [current_design]



et_property IOSTANDARD LVCMOS33 [get_ports {emio_tri_io[*]}]

pl led

et_property PACKAGE_PIN AE15 [get_ports {emio_tri_io[0]}]

pl key

et_property PACKAGE_PIN AE14 [get_ports {emio_tri_io[1]}]
```

8.

Generate bit file

9.

Although the logic of the PL end is not used, the pins of the PL end are used.

Therefore, when exporting hardware, select "Include bitstream".

3.

Serial port print information

4.

Summary of this chapter

This chapter introduces the use of two RS485 circuits on the board for loop testing. You can expand the functionality for testing based on this.

Part 15: PL-side Ethernet experiment

The experimental Vivado project directory is "pl_net/vivado".

The experimental vitis project directory is "pl_net/vitis".

The usage of Ethernet on the PS side was introduced earlier, and this chapter introduces Ethernet on the PL side.

Job responsibilities of FPGA engineer

The following is the content that FPGA engineers are responsible for.

1.

Hardware environment setup

1.

Configure interrupts and AXI HP0 FPDs based on "ps_hello"

```
include "xil_printf.h"

include "ax_pwm.h"

include "xil_io.h"

include "xparameters.h"

include "sleep.h"



nsignedint duty;



ıt main()
```

```
ıit_platform();



rint("Hello World\n\r");



pwm out period = frequency(pwm_out) * (2^N) / frequency(clk);

.X_PWM_mWriteReg(XPAR_AX_PWM_0_S00_AXI_BASEADDR,

.X_PWM_S00_AXI_SLV_REG0_OFFSET,17179);//200hz

duty = (2^N) * (1 - (duty cycle)) - 1

/hile(1){

›r(duty =0x8fffffff; duty <0xffffffff; duty = duty +100000){

.X_PWM_mWriteReg(XPAR_AX_PWM_0_S00_AXI_BASEADDR,

.X_PWM_S00_AXI_SLV_REG1_OFFSET, duty);
```

```
sleep(100);

leanup_platform();

eturn0;
```

4.

By running the code, we can see that PLLED presents a breathing light effect.

5.

Let's check the registers through debugging

2.

Add reset module for reset

You can see that the newly added IP appears.

4.

Connect the BRAM_PORTA of AXI BRAM Controller to PORTA of BRAM, connect the BRAM_PORT of pl_bram_ctrl to PORTB of BRAM. Connect the interrupt signal intr of pl_bram_ctrl module to the interrupt port of ZYNQ. And click Auto Connect

5.

Select the BRAM address size in the Address Editor, such as setting 4K space, to query the BRAM space to 1K depth.

1.1

Block Design Adding Logic Analyzer Method

6.

Another way to add a logic analyzer is to select BRAM_PORT bus and intr interrupt, right-click and select Debug.

3.

After adding, you can see the IP of the dynamic clock.

4.

Add a dynamic clock controller, the main function of this module is to configure different clock outputs according to different resolutions, essentially calling the phase-locked loop, but it should be noted that the reference clock of this module must be set to 100MHz

And connect the clock

2.

Add mipi_csi2_rx_subsystem module for receiving and parsing MIPI data, and convert it to axi-stream interface. The configuration is as follows: select RAW10 for data format, select 2 lanes, and configure Line Rate to 1000Mbps, which refers to the maximum supported rate. You can also fill in according to your own needs, with a range of 80-2500. The default configuration of Pixels Per Clock is 1, which means 1 pixel per cycle.

ob responsibilities of software engineers

The following is the content that software engineers are responsible for.

Note: Due to the use of lwip, it is necessary to modify the lwip library before conducting this experiment. Please refer to the chapter "Lwip for PS Ethernet Use".

2.

Vitis program development

2.1

PL-side Ethernet test

The establishment method of Vitis project is the same as that of PS side, and the process will not be repeated. The lwip library has been modified earlier. The testing method has been introduced earlier.

And the dual-core agreement has a shared address, so that parameters can be passed.

3.

In the while loop, it is determined that there is a software interrupt from CPU1, and the string in CPU1 Memory Space is printed out.

1.4

CPU1 Program Introduction

1.

There is also a character array in the CPU1 program, and Cpu0Data points to the string address of the CPU0 Memory Space.

2.

In the PLGpioSetup function, the key interrupt number needs to be bound to CPU1.

Other parts are similar to CPU0 and will not be repeated.

2.

On board verification

1.1

Block Design Adding Logic Analyzer Method

6.

Another way to add a logic analyzer is to select BRAM_PORT bus and intr interrupt, right-click and select Debug.

7.

You can see that there are small insects on the bus. Click Run Connection Automation to automatically connect

5.

Export pin

6.

Save the design, press F6 to check the design, and Generate Output Products...

7.

Since the software automatically binds the pins in reverse order from the hardware design, it is necessary to modify the pin binding. Expand the red box below in IP Sources

4.

Save Designs and Generate Output Products

If you want to add a custom IP to the IP library, click IP Catalog and right-click Add Repository in Vivado Repository.

Find the folder where the custom IP is located and click Select.

Jump out of the window, select IP, and click OK.